

Week 2 – PL/SQL Programming

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

PL/SQL Code:

```
BEGIN

  FOR cust IN (
    SELECT CustomerID, DOB
    FROM Customers
  )
  LOOP
    IF MONTHS_BETWEEN(SYSDATE, cust.DOB)/12 > 60 THEN

      UPDATE Loans
      SET InterestRate = InterestRate - 1
      WHERE CustomerID = cust.CustomerID;

    END IF;
  END LOOP;

  COMMIT;

END;
```

```
SELECT CustomerID, InterestRate
```

```
FROM Loans;
```

Output Screenshot

	CUSTOMERID	INTERESTRATE
1	1	4
2	2	7

Scenario 2: A customer can be promoted to VIP status based on their balance.

Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

PL/SQL Code:

```
ALTER TABLE Customers
```

```
ADD IsVIP VARCHAR2(5);
```

```
BEGIN
```

```
  FOR cust IN (
```

```
    SELECT CustomerID, Balance
```

```
    FROM Customers
```

```
  )
```

```
  LOOP
```

```
    IF cust.Balance > 10000 THEN
```

```
      UPDATE Customers
```

```

        SET IsVIP = 'TRUE'

        WHERE CustomerID = cust.CustomerID;

    END IF;

END LOOP;

COMMIT;

END;

/

SELECT CustomerID, Name, Balance, IsVIP
FROM Customers;

```

Output Screenshot

	CUSTOMERID	NAME	BALANCE	ISVIP
1	1	John Doe	1000	(null)
2	2	Jane Smith	1500	(null)

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

PL/SQL Code:

```

SELECT CustomerID, LoanID, EndDate
FROM Loans
WHERE EndDate BETWEEN SYSDATE AND SYSDATE + 30;

```

Output Screenshot

	CUSTOMERID	LOANID	ENDDATE
1	1	1	7/21/2026, 4:47:28

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

PL/SQL Code:

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (  
    p_FromAccount IN NUMBER,  
    p_ToAccount IN NUMBER,  
    p_Amount IN NUMBER  
)  
IS  
    v_Balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_Balance  
    FROM Accounts  
    WHERE AccountID = p_FromAccount;  
    IF v_Balance < p_Amount THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');  
    END IF;  
    UPDATE Accounts  
    SET Balance = Balance - p_Amount  
    WHERE AccountID = p_FromAccount;  
    UPDATE Accounts  
    SET Balance = Balance + p_Amount  
    WHERE AccountID = p_ToAccount;
```

```

COMMIT;

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

/

SET SERVEROUTPUT ON;

BEGIN

    SafeTransferFunds(1,2,500);

END;

/

SELECT AccountID,Balance

FROM Accounts;

```

Output Screenshot 1

Download Execution time: 0.005 seconds

	ACCOUNTID	BALANCE
1	1	0
2	2	2500

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

PL/SQL Code:

```

CREATE OR REPLACE PROCEDURE UpdateSalary (

    p_EmployeeID IN NUMBER,

```

```

    p_Percentage IN NUMBER
)
IS
BEGIN

    UPDATE Employees

    SET Salary = Salary + (Salary * p_Percentage /100)

    WHERE EmployeeID = p_EmployeeID;

    IF SQL%ROWCOUNT = 0 THEN

        RAISE_APPLICATION_ERROR(-20002,'Employee ID does not exist');

    END IF;

    COMMIT;

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE(SQLERRM);

END;

/

SET SERVEROUTPUT ON;


BEGIN

    UpdateSalary(1,10);

END;

/

SELECT EmployeeID,Name,Salary

FROM Employees;

```

Output Screenshot 1

	EMPLOYEEID	NAME	SALARY
1	1	Alice Johnson	84700
2	2	Bob Brown	60000

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

PL/SQL Code:

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(  
  
    p_CustomerID NUMBER,  
  
    p_Name VARCHAR2,  
  
    p_DOB DATE,  
  
    p_Balance NUMBER  
  
)  
  
IS  
  
BEGIN  
  
    INSERT INTO Customers  
  
    VALUES( p_CustomerID, p_Name, p_DOB, p_Balance,SYSDATE,NULL );  
  
    COMMIT;  
  
EXCEPTION  
  
    WHEN DUP_VAL_ON_INDEX THEN  
  
        ROLLBACK;  
  
        DBMS_OUTPUT.PUT_LINE('Customer ID already exists.');
```

```
    WHEN OTHERS THEN  
  
        ROLLBACK;
```

```

        DBMS_OUTPUT.PUT_LINE(SQLERRM);

END;

/

SET SERVEROUTPUT ON;

BEGIN

    AddNewCustomer(

        3,

        'David Miller',

        TO_DATE('1998-06-10','YYYY-MM-DD'),

        9000

    );

END;

/

SELECT CustomerID,Name,Balance

FROM Customers;

```

Output Screenshot 1

	CUSTOMERID	NAME	BALANCE
1	3	David Miller	9000
2	1	John Doe	1000
3	2	Jane Smith	1500

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

PL/SQL Code:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
```

```
IS
```

```
BEGIN
```

```
    UPDATE Accounts
```

```
    SET Balance = Balance + (Balance * 0.01)
```

```
    WHERE AccountType = 'Savings';
```

```
    COMMIT;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    ProcessMonthlyInterest;
```

```
END;
```

```
/
```

```
SELECT AccountID, AccountType, Balance
```

```
FROM Accounts;
```

Output Screenshot 1

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	1	Savings	0
2	2	Checking	2500

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

PL/SQL Code:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
```

```
    p_Department IN VARCHAR2,
```

```
    p_BonusPercent IN NUMBER
```

```
)
```

```
IS
```

```
BEGIN
```

```
    UPDATE Employees
```

```
    SET Salary = Salary + (Salary * p_BonusPercent / 100)
```

```
    WHERE Department = p_Department;
```

```
    COMMIT;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    UpdateEmployeeBonus('HR',10);
```

```
END;
```

```
/
```

```
SELECT EmployeeID, Name, Department, Salary
```

```
FROM Employees;
```

Output Screenshot 1

	EMPLOYEEID	NAME	DEPARTMENT	SALARY
1	1	Alice Johnson	HR	93170
2	2	Bob Brown	IT	60000

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

PL/SQL Code:

```
BEGIN
```

```
    TransferFunds(1,2,2000);
```

```
END;
```

```
/
```

```
SELECT AccountID, Balance
```

```
FROM Accounts;
```

Output Screenshot

	ACCOUNTID	BALANCE
1	1	8000
2	2	12000

Exercise 4: Functions

Scenario 1: Calculate the age of customers for eligibility checks.

- **Question:** Write a function **CalculateAge** that takes a customer's date of birth as input and returns their age in years.

PL/SQL Code:

```
CREATE OR REPLACE FUNCTION CalculateAge (
```

```
    p_DOB DATE
```

```
)
```

```
RETURN NUMBER
```

```
IS
```

```
    v_Age NUMBER;
```

```
BEGIN
```

```

v_Age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_DOB) / 12);

RETURN v_Age;

END;

/

SELECT CustomerID,

       Name,

       CalculateAge(DOB) AS Age

FROM Customers;

```

Output Screenshot

	CUSTOMERID	NAME	AGE
1	3	David Miller	28
2	1	John Doe	71
3	2	Jane Smith	35

Scenario 2: The bank needs to compute the monthly installment for a loan.

- **Question:** Write a function **CalculateMonthlyInstallment** that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

PL/SQL Code:

```

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(

    p_LoanAmount NUMBER,

    p_InterestRate NUMBER,

    p_Years NUMBER

)

RETURN NUMBER

IS

    v_MonthlyInstallment NUMBER;

BEGIN

```

```

v_MonthlyInstallment :=
    (p_LoanAmount + (p_LoanAmount * p_InterestRate / 100))
    / (p_Years * 12);

RETURN ROUND(v_MonthlyInstallment,2);

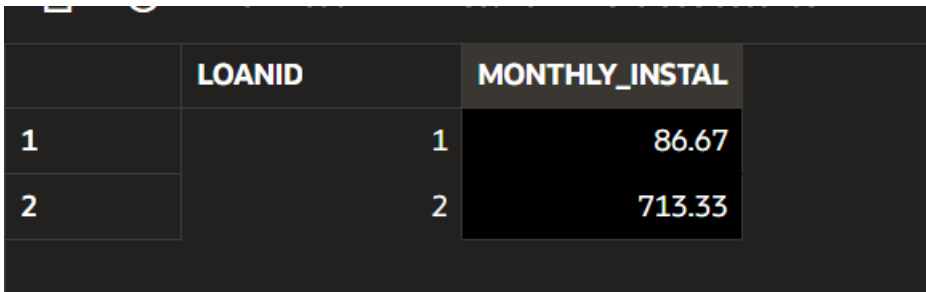
END;

/

SELECT LoanID,
       CalculateMonthlyInstallment(
           LoanAmount,
           InterestRate,
           5
       ) AS Monthly_Installment
FROM Loans;

```

Output Screenshot



	LOANID	MONTHLY_INSTALL
1	1	86.67
2	2	713.33

Scenario 3: Check if a customer has sufficient balance before making a transaction.

- **Question:** Write a function **HasSufficientBalance** that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

PL/SQL Code:

```

CREATE OR REPLACE FUNCTION HasSufficientBalance(
    p_AccountID NUMBER,
    p_Amount NUMBER
)

```

```

RETURN VARCHAR2

IS

    v_Balance NUMBER;

BEGIN

    SELECT Balance

    INTO v_Balance

    FROM Accounts

    WHERE AccountID = p_AccountID;

    IF v_Balance >= p_Amount THEN

        RETURN 'TRUE';

    ELSE

        RETURN 'FALSE';

    END IF;

END;

/

SELECT AccountID,

       Balance,

       HasSufficientBalance(AccountID,5000) AS Status

FROM Accounts;

```

Output Screenshot

	ACCOUNTID	BALANCE	STATUS
1	1	8000	TRUE
2	2	12000	TRUE

Exercise 5: Triggers

Scenario 1: Automatically update the last modified date when a customer's record is updated.

- **Question:** Write a trigger **UpdateCustomerLastModified** that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

PL/SQL Code:

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
```

```
BEFORE UPDATE ON Customers
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    :NEW.LastModified := SYSDATE;
```

```
END;
```

```
/
```

```
UPDATE Customers
```

```
SET Balance = Balance + 500
```

```
WHERE CustomerID = 1;
```

```
COMMIT;
```

```
SELECT CustomerID, Name, Balance, LastModified
```

```
FROM Customers;
```

Output Screenshot

	CUSTOMERID	NAME	BALANCE	LASTMODIFIED
1	3	David Miller	9000	7/1/2026, 5:13:00 P
2	1	John Doe	1500	7/1/2026, 5:40:27 P
3	2	Jane Smith	1500	7/1/2026, 4:13:54 P

Scenario 2: Maintain an audit log for all transactions.

- **Question:** Write a trigger **LogTransaction** that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

PL/SQL Code:

```
CREATE TABLE AuditLog (  
    LogID NUMBER GENERATED BY DEFAULT AS IDENTITY,  
    TransactionID NUMBER,  
    AccountID NUMBER,  
    Amount NUMBER,  
    TransactionType VARCHAR2(20),  
    LogDate DATE  
);
```

```
CREATE OR REPLACE TRIGGER LogTransaction  
AFTER INSERT ON Transactions  
FOR EACH ROW  
BEGIN  
    INSERT INTO AuditLog  
    (  
        TransactionID,  
        AccountID,  
        Amount,  
        TransactionType,  
        LogDate  
    )  
VALUES  
    (  
        
```



```

:NEW.TransactionID,

:NEW.AccountID,

:NEW.Amount,

:NEW.TransactionType,

SYSDATE

);

END;

/

```

```

INSERT INTO Transactions
VALUES
(
3,
1,
SYSDATE,
500,
'Deposit'
);

```

```

COMMIT;

```

```

SELECT *
FROM AuditLog;

```

Output Screenshot

	LOGID	TRANSACTIONID	ACCOUNTID	AMOUNT	TRANSACTIONTYF	LOGDATE
1	1	3	1	500	Deposit	7/1/2026, 5:58:47 P

Scenario 3: Enforce business rules on deposits and withdrawals.

- **Question:** Write a trigger **CheckTransactionRules** that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

PL/SQL Code:

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
```

```
BEFORE INSERT ON Transactions
```

```
FOR EACH ROW
```

```
DECLARE
```

```
    v_Balance NUMBER;
```

```
BEGIN
```

```
    SELECT Balance
```

```
    INTO v_Balance
```

```
    FROM Accounts
```

```
    WHERE AccountID = :NEW.AccountID;
```

```
    IF :NEW.TransactionType = 'Withdrawal'
```

```
        AND :NEW.Amount > v_Balance THEN
```

```
        RAISE_APPLICATION_ERROR(-20001,
```

```
        'Insufficient Balance');
```

```
    ELSIF :NEW.TransactionType = 'Deposit'
```

```
        AND :NEW.Amount <= 0 THEN
```

```
        RAISE_APPLICATION_ERROR(-20002,
```

```
        'Deposit amount must be positive');
```

END IF;

END;

/

INSERT INTO Transactions

VALUES

(

4,

1,

SYSDATE,

20000,

'Withdrawal'

);

SELECT *

FROM Transactions;

Output Screenshot

	TRANSACTIONID	ACCOUNTID	TRANSACTIONDATE	AMOUNT	TRANSACTIONTYPE
1	1	1	7/1/2026, 4:14 PM	200	Deposit
2	2	2	7/1/2026, 4:14 PM	300	Withdrawal
3	3	1	7/1/2026, 5:58:47 P	500	Deposit

Exercise 6: Cursors

Scenario 1: Generate monthly statements for all customers.

- **Question:** Write a PL/SQL block using an explicit cursor **GenerateMonthlyStatements** that retrieves all transactions for the current month and prints a statement for each customer.

PL/SQL Code:

```
SET SERVEROUTPUT ON;

DECLARE

    CURSOR GenerateMonthlyStatements IS

        SELECT TransactionID,

               AccountID,

               TransactionDate,

               Amount,

               TransactionType

        FROM Transactions

        WHERE TO_CHAR(TransactionDate,'MMYYYY') =

              TO_CHAR(SYSDATE,'MMYYYY');

    v_Transaction GenerateMonthlyStatements%ROWTYPE;

BEGIN

    OPEN GenerateMonthlyStatements;

    LOOP

        FETCH GenerateMonthlyStatements INTO v_Transaction;

        EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(

            'Transaction ID: ' || v_Transaction.TransactionID ||
```

```

        ', Account: ' || v_Transaction.AccountID ||
        ', Amount: ' || v_Transaction.Amount ||
        ', Type: ' || v_Transaction.TransactionType
    );
END LOOP;

CLOSE GenerateMonthlyStatements;

END;

/

```

Output Screenshot

```

Transaction ID: 1, Account: 1, Amount: 200, Type: Deposit
Transaction ID: 2, Account: 2, Amount: 300, Type: Withdrawal
Transaction ID: 3, Account: 1, Amount: 500, Type: Deposit

PL/SQL procedure successfully completed.

```

Scenario 2: Apply annual fee to all accounts.

- **Question:** Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

PL/SQL Code:

```

DECLARE

CURSOR ApplyAnnualFee IS

    SELECT AccountID

    FROM Accounts;

BEGIN

FOR acc IN ApplyAnnualFee LOOP

    UPDATE Accounts

    SET Balance = Balance - 100

    WHERE AccountID = acc.AccountID;

```

```
END LOOP;
```

```
COMMIT;
```

```
END;
```

```
/
```

```
SELECT AccountID, Balance
```

```
FROM Accounts;
```

Output Screenshot

	ACCOUNTID	BALANCE
1	1	7800
2	2	11800

Scenario 3: Update the interest rate for all loans based on a new policy.

- **Question:** Write a PL/SQL block using an explicit cursor **UpdateLoanInterestRates** that fetches all loans and updates their interest rates based on the new policy.

PL/SQL Code:

```
DECLARE
```

```
    CURSOR UpdateLoanInterestRates IS
```

```
        SELECT LoanID
```

```
        FROM Loans;
```

```
BEGIN
```

```
    FOR loan IN UpdateLoanInterestRates LOOP
```

```
        UPDATE Loans
```

```
        SET InterestRate = InterestRate + 0.5
```

```
        WHERE LoanID = loan.LoanID;
```

```
    END LOOP;
```

```
COMMIT;
```

```
END;
```

```
/
```

```
SELECT LoanID,
```

```
       InterestRate
```

```
FROM Loans;
```

Output Screenshot

	LOANID	INTERESTRATE
1	1	4.5
2	2	7.5

Exercise 7: Packages

Scenario 1: Group all customer-related procedures and functions into a package.

- **Question:** Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

PL/SQL Code:

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
```

```
    PROCEDURE AddCustomer(  
        p_CustomerID NUMBER,  
        p_Name VARCHAR2,  
        p_DOB DATE,  
        p_Balance NUMBER  
    );
```

```

PROCEDURE UpdateCustomer(
    p_CustomerID NUMBER,
    p_Balance NUMBER
);

FUNCTION GetCustomerBalance(
    p_CustomerID NUMBER
) RETURN NUMBER;

END CustomerManagement;

/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(
        p_CustomerID NUMBER,
        p_Name VARCHAR2,
        p_DOB DATE,
        p_Balance NUMBER
    )
    IS
    BEGIN
        INSERT INTO Customers
        VALUES(
            p_CustomerID,
            p_Name,
            p_DOB,
            p_Balance,
            SYSDATE,
            NULL
        );
    END;

END;

```



```
PROCEDURE UpdateCustomer(  
    p_CustomerID NUMBER,  
    p_Balance NUMBER  
)  
IS  
BEGIN  
    UPDATE Customers  
    SET Balance = p_Balance  
    WHERE CustomerID = p_CustomerID;  
END;
```

```
FUNCTION GetCustomerBalance(  
    p_CustomerID NUMBER  
)  
RETURN NUMBER  
IS  
    v_Balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_Balance  
    FROM Customers  
    WHERE CustomerID = p_CustomerID;  
  
    RETURN v_Balance;  
END;
```

```
END CustomerManagement;
```

/

BEGIN

CustomerManagement.UpdateCustomer(1,15000);

END;

/

SELECT CustomerID, Name, Balance

FROM Customers;

Output Screenshot

	CUSTOMERID	NAME	BALANCE
1	3	David Miller	9000
2	1	John Doe	15000
3	2	Jane Smith	1500

Scenario 2: Create a package to manage employee data.

- **Question:** Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

PL/SQL Code:

CREATE OR REPLACE PACKAGE EmployeeManagement AS

PROCEDURE HireEmployee(
p_ID NUMBER,
p_Name VARCHAR2,
p_Position VARCHAR2,
p_Salary NUMBER,
p_Department VARCHAR2,

```
    p_HireDate DATE
);
```

```
PROCEDURE UpdateEmployee(
    p_ID NUMBER,
    p_Salary NUMBER
);
```

```
FUNCTION AnnualSalary(
    p_ID NUMBER
) RETURN NUMBER;
```

```
END EmployeeManagement;
```

```
/
```

```
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
```

```
    PROCEDURE HireEmployee(
        p_ID NUMBER,
        p_Name VARCHAR2,
        p_Position VARCHAR2,
        p_Salary NUMBER,
        p_Department VARCHAR2,
        p_HireDate DATE
    )
```

```
    IS
```

```
    BEGIN
```

```
        INSERT INTO Employees
```

```
VALUES(  
    p_ID,  
    p_Name,  
    p_Position,  
    p_Salary,  
    p_Department,  
    p_HireDate  
);  
END;
```

```
PROCEDURE UpdateEmployee(  
    p_ID NUMBER,  
    p_Salary NUMBER  
)  
IS  
BEGIN  
    UPDATE Employees  
    SET Salary = p_Salary  
    WHERE EmployeeID = p_ID;  
END;
```

```
FUNCTION AnnualSalary(  
    p_ID NUMBER  
)  
RETURN NUMBER  
IS  
    v_Salary NUMBER;  
BEGIN
```

```

SELECT Salary
INTO v_Salary
FROM Employees
WHERE EmployeeID = p_ID;

RETURN v_Salary * 12;

END;

```

```

END EmployeeManagement;

```

```

/

```

```

SELECT EmployeeManagement.AnnualSalary(1) AS Annual_Salary
FROM Dual;

```

Output Screenshot



	ANNUAL_SALARY
1	1118040

Scenario 3: Group all account-related operations into a package.

- **Question:** Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

PL/SQL Code:

```

CREATE OR REPLACE PACKAGE AccountOperations AS

```

```

PROCEDURE OpenAccount(
    p_AccountID NUMBER,

```

```
    p_CustomerID NUMBER,  
    p_AccountType VARCHAR2,  
    p_Balance NUMBER  
);
```

```
PROCEDURE CloseAccount(  
    p_AccountID NUMBER  
);
```

```
FUNCTION TotalBalance(  
    p_CustomerID NUMBER  
) RETURN NUMBER;
```

```
END AccountOperations;
```

```
/
```

```
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
```

```
    PROCEDURE OpenAccount(  
        p_AccountID NUMBER,  
        p_CustomerID NUMBER,  
        p_AccountType VARCHAR2,  
        p_Balance NUMBER  
    )
```

```
    IS
```

```
    BEGIN
```

```
        INSERT INTO Accounts
```

```
        VALUES(  

```

```

        p_AccountID,
        p_CustomerID,
        p_AccountType,
        p_Balance,
        SYSDATE
    );
END;

PROCEDURE CloseAccount(
    p_AccountID NUMBER
)
IS
BEGIN
    DELETE FROM Accounts
    WHERE AccountID = p_AccountID;
END;

FUNCTION TotalBalance(
    p_CustomerID NUMBER
)
RETURN NUMBER
IS
    v_Total NUMBER;
BEGIN
    SELECT SUM(Balance)
    INTO v_Total
    FROM Accounts
    WHERE CustomerID = p_CustomerID;

```

```
RETURN NVL(v_Total,0);  
END;
```

```
END AccountOperations;
```

```
/
```

```
SELECT AccountOperations.TotalBalance(1) AS Total_Balance  
FROM Dual;
```

Output Screenshot

	TOTAL_BALANCE
1	7800